Table of Contents

1. Executive Summary . . . 3

2. Product Overview . . . 4

2.1 The problem BatteryLab solves . . . 4

2.2 Feature map . . . 4

2.3 Design principles . . . 5

3. Getting Started . . . 6

3.1 Prerequisites and installation . . . 6

3.2 Interface orientation . . . 6

3.3 A five-minute quick start . . . 7

4. User Guide - The Seven-Step Workflow . . . 8

4.1 Step 1 · Design - define the cell . . . 8

4.2 Step 2 · Connection - size and wire the pack . . . 9

4.3 Step 3 · Modeling - the finished pack in 3D . . . 9

4.4 Step 4 · Simulation - run the electrical model . . . 10

4.5 Step 5 · Thermal - heat, limits and cooling . . . 12

4.6 Step 6 · Monitor - BMS telemetry, simulated and real . . . 12

4.7 Step 7 · Analysis - compare, export, conclude . . . 13

4.8 Library - the datasheet browser . . . 14

5. VoltAI Copilot . . . 15

6. System Architecture . . . 16

6.1 Technology stack . . . 17

6.2 Repository layout . . . 18

6.3 Client state model . . . 18

6.4 3D rendering pipeline . . . 19

6.5 Shared playback clock . . . 19

6.6 Design system and motion . . . 19

7. Physics and Simulation Engine . . . 20

7.1 Open-circuit voltage model . . . 20

7.2 Equivalent circuit and state update . . . 20
7.3 Thermal model . . . 20
7.4 Test profiles . . . 21
7.5 Numerical scheme and outputs . . . 21
7.6 Safety checks . . . 22
7.7 Validation performed . . . 22
7.8 Assumptions and limitations . . . 22
8. Data Layer and API Reference . . . 23
8.1 Database schema . . . 23
8.2 API endpoints . . . 23
8.3 The simulate pipeline in detail . . . 24
8.4 Copilot context and patch sanitisation . . . 24
8.5 Exports and data escape hatches . . . 25
9. Frontend Engineering and Quality . . . 25
9.1 Responsiveness . . . 25
9.2 Accessibility and theming . . . 25
9.3 Verification methodology . . . 25
9.4 Known constraints . . . 26
10. Extensibility and Roadmap . . . 26
10.1 Extension points that exist today . . . 26
10.2 Candidate roadmap . . . 26
10.3 Deployment notes . . . 27
10.4 Closing assessment . . . 27

How to read this report

Chapters 1 through 3 introduce the product and help a new user reach their first simulation in minutes. Chapters 4 and 5 are the operational heart of the document: a step-by-step walkthrough of the seven workflow tabs and the VoltAI copilot, with annotated screenshots of a real session. Chapters 6 through 9 are written for engineers and reviewers: they document the system architecture, the physics engine and its equations, the data layer and API contract, and the frontend engineering practices used to keep the app reliable. Chapter 10 closes with extensibility notes and a candid roadmap. All screenshots come from a live session on the shipped build, and every quoted number originates from that session.

1. Executive Summary

BatteryLab is a browser-based virtual battery engineering platform that compresses the traditional cell-selection, pack-design and thermal-testing cycle into a single guided workspace. Instead of ordering cells, wiring a bench and running a cycler, an engineer opens the website, defines a cell from a curated datasheet library or from scratch, assembles it into a series-parallel pack, and then runs electro-thermal experiments that behave like real bench tests: constant-current discharges, CC-CV charges, HPPC pulse sequences and an urban drive cycle, all animated on a shared playback clock with live charts, safety verdicts and exportable datasets.

The application is organised as a seven-step wizard - Design, Connection, Modeling, Simulation, Thermal, Monitor and Analysis - reinforced by a 24-cell datasheet Library and an always-available AI copilot named VoltAI. Each step feeds the next: the electrical parameters entered in Design drive the schematic in Connection, which drives the 3D pack geometry in Modeling, the test profiles in Simulation, the heat model in Thermal, the telemetry dashboard in Monitor and the cross-run comparisons in Analysis. Wizard gating keeps the sequence honest - the Connection step cannot be marked complete until the schematic is physically wired, and every completed step earns a checkmark in the header navigation.

7	24	8	5
workflow steps, one shared session	datasheet cells in the library	supported chemistries	test profiles incl. custom

Under the hood the platform is a Next.js 16 single-page application built on React 19, TypeScript, Tailwind CSS 4 and shadcn/ui, with state held in a lightweight zustand store. Physics runs on a deterministic 2-RC equivalent-circuit model coupled to a Bernardi heat-generation equation and a lumped thermal mass, integrated at one-second steps in TypeScript on the server. Results persist to a SQLite database through Prisma 6, so runs survive reloads and can be compared, replayed and exported as CSV at any time. Interactive 3D - the finished pack, its busbars and BMS, and the live thermal field - is rendered with react-three-fiber and three.js, while every 2D chart uses recharts.

This report serves two audiences at once. For users, Chapters 2 to 5 explain what the platform does, how to install and launch it, and how to drive every screen, including the hardware monitoring mode that streams telemetry from a physical battery management system over CAN, RS485 or UART. For engineers, Chapters 6 to 9 document the architecture, the equations and numerical scheme behind the simulation, the API and database contract, and the quality practices - automated browser walkthroughs, zero-console-error verification, responsive and accessibility checks - that keep the platform dependable. Chapter 10 closes with extension points and a roadmap of candidate features.

2. Product Overview

2.1 The problem BatteryLab solves

Designing a battery pack is a discipline of trade-offs: chemistry versus cost, energy versus power, cooling complexity versus temperature margins, cell count versus voltage targets. Physical testing of every candidate configuration is slow, expensive and sometimes dangerous - a mis-configured adiabatic test can push a lithium-ion cell past the 130 degree Celsius runaway onset in minutes. Virtual prototyping shortens that loop, but most simulation tools are either heavyweight desktop packages with steep learning curves or thin calculators that hide the physics behind a single number. BatteryLab occupies the middle ground: it is instantly accessible in a browser, yet every result traces back to documented engineering models that the user can inspect and edit.

The platform was built around three commitments. First, **traceability**: the same 44-field cell definition - identity, chemistry, electrical ratings, ECM parameters, geometry, thermal properties, temperature windows - flows unchanged from the Design sheet into the schematic, the 3D model, the simulation, the thermal test and the stored run, so a number seen in Analysis can always be traced back to its datasheet origin. Second, **honest physics**: terminal voltage sags below OCV during discharge, the CC-CV taper clamps exactly at V-max and ends at C/20, and an uncooled 4C test will abort at the 130 degree Celsius runaway threshold. Third, **editing without lock-in**: loading a library cell only pre-fills the design sheet; every value remains editable, and the engine accepts custom values as gladly as datasheet ones.

2.2 Feature map

The table below summarises what each part of the workspace does and the artefacts it produces. The workflow reads top to bottom, but nothing prevents an engineer from jumping back - changing the series count in Connection immediately resizes the 3D pack in Modeling and re-scales pack voltage everywhere.

Workspace area	Purpose	Key outputs
1 · Design	Define or load the cell: identity, chemistry, electrical ratings, ECM parameters, geometry, thermal data, pack targets	Valid 44-field design sheet, live KPI preview, saved design rows
2 · Connection	Size the pack: series/parallel steppers, 12/24/48/400 V presets, interactive schematic builder	S×P configuration, animated wiring diagram, gated completion
3 · Modeling	Inspect the finished pack mechanically in interactive 3D with busbars, BMS board and enclosure	3D pack view, mechanical specification strip, spec CSV / snapshot exports via Analysis

Workspace area	Purpose	Key outputs
4 · Simulation	Run electrical test profiles against the 2-RC model and watch an animated bench replay	Voltage/SOC/current/temperature charts, 12-metric summary, safety checks, CSV export
5 · Thermal	Apply cooling strategies and test heat rejection against datasheet limits	Thermal verdict banner, 2D pack heat map, 3D thermal field, temperature-vs-limits chart
6 · Monitor	Watch telemetry like a BMS dashboard: gauges, per-cell matrix, contactors, alerts	Replay of stored runs; live stream from real hardware over CAN / RS485 / UART
7 · Analysis	Compare every stored run side by side and export the evidence	Program KPIs, multi-run overlays, energy and delta-T bars, final-pack 3D with exports
Library	Browse 24 real datasheet cells across 17 brands and 8 chemistries	One-click "Use in Design" pre-fill of the whole design sheet
VoltAI copilot	Ask an AI engineer that sees the live design, pack, test parameters and last result	Explanations, recommendations, one-click Apply patches to the design

Table 2-1 · Workspace areas and their outputs

2.3 Design principles

Several engineering principles shaped the product. **Datasheet-first**: the cell library stores real headline specifications - capacity, voltage window, internal resistance, mass, C-rates, cycle life - so designs start from reality, and the RC-network values that datasheets rarely publish are clearly presented as editable engineering starting points. **One clock**: Simulation, Thermal and Monitor all replay through a single playback engine, so the cycler animation, the gauges, the cell matrix and the scrolling charts always agree on "now". **Safety by default**: every run ends with PASS / WARN / FAIL badges against C-rate limits, the voltage window, the 45 / 60 degree Celsius temperature bands and the 130 degree Celsius runaway onset, and the thermal tab computes a plain-language verdict before any hardware is touched.

The interface itself follows a professional restraint: a single blue colour family, one accent gradient, glass-effect navigation, staggered card entrances and hover micro-interactions implemented purely in CSS so that no animation library bloats the bundle, and a **prefers-reduced-motion** guard that disables all non-essential motion for users who request it. Light and dark themes are first-class citizens - both are fully supported across every tab, including the 3D views - and the layout holds together from a 390 pixel phone to a wide desktop without horizontal overflow.

3. Getting Started

3.1 Prerequisites and installation

BatteryLab is a standard Next.js 16 application. To run it locally you need Node.js 18 or newer (the project is developed with the bun runtime, which also works for every command below) and a Chromium-based browser - Chrome or Edge - to unlock the Web Serial hardware mode in the Monitor step; every other feature works in any modern browser including Firefox and Safari. The database engine is embedded: SQLite lives in a single file at db/custom.db, so there is no database server to install.

Setup and launch

```
git clone <project>  &&  cd batterylab
bun install                    # or: npm install
bunx prisma generate           # generate the Prisma client
bunx prisma db push            # create / update db/custom.db
bun run dev                    # start on http://localhost:3000
```

The dev server listens on port 3000 and hot-reloads. For a production-style run, **bun run build** produces a standalone server bundle which is then started with **bun run start**; the SQLite file carries over untouched, preserving every saved design and test run. On first launch the database seeds a demonstration design ("NMC 21700 EV Cell") and a small set of curated runs so that the Analysis tab has something to compare immediately; deleting these demo rows is safe.

3.2 Interface orientation

Everything happens on a single page. A glass-effect header carries the BatteryLab brand mark, the seven numbered workflow buttons, a Library button and a theme toggle; a thin gradient hairline under the header tracks reading progress. Each step displays a numbered icon chip, a title and a one-line brief, followed by cards whose entrance animations cascade in sequence. As the user makes progress, the number badge on a completed step flips to a green checkmark - Design completes when the sheet passes validation, Connection when the schematic is fully wired, Simulation when a run has been stored - so the header doubles as a progress tracker.

Two floating elements are available on every tab. The theme toggle (top right) switches between the light and dark palettes with an animated icon. The VoltAI launcher (bottom right, a gradient pill with a glow) opens the AI copilot panel; Section 5 covers it in depth. A quiet footer reminds the user that the models are engineering estimates and should be validated with physical testing - a deliberate, honest caveat that the platform keeps visible at all times.

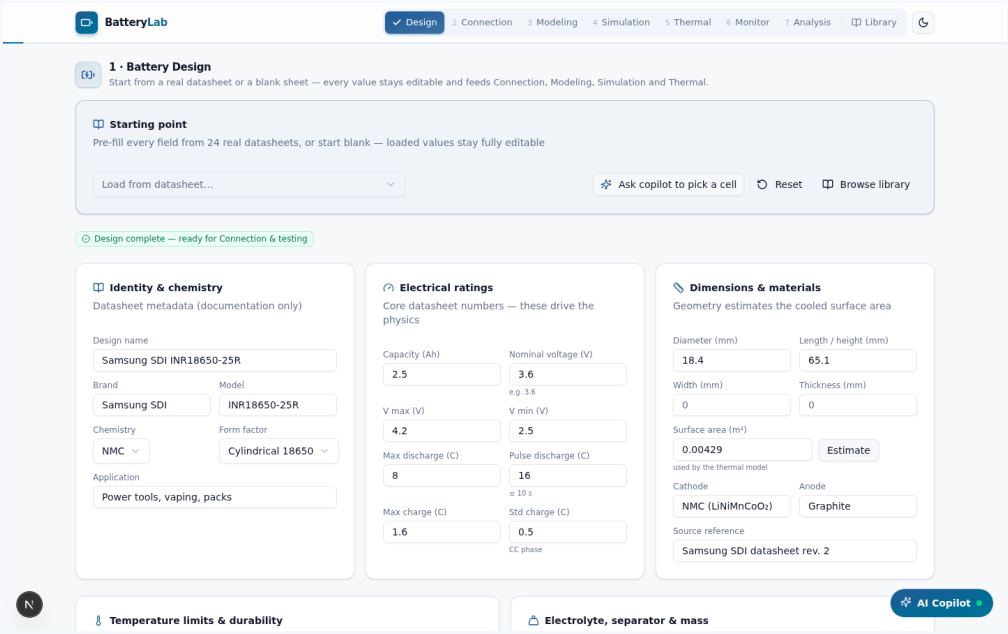


Figure 3-1 · The workspace in light mode: Design step with a datasheet cell loaded, completion badge, checkmarks in the nav

3.3 A five-minute quick start

The fastest meaningful tour takes about five minutes and exercises the full pipeline. (1) Open **Library**, pick "Samsung SDI INR18650-25R" and press "Use in Design" - the Design tab fills with the datasheet values and shows the green "Design complete" badge. (2) On Design, set the pack targets to 48 V and 10 Ah; the sheet recommends 13S4P. (3) Move to **Connection**, press "Connect parallel", "Connect series", then "Connect all" to wire the schematic, which unlocks the Next button. (4) Open **Modeling** and orbit the 3D pack - notice the red positive caps, the dark negative rings and the BMS board. (5) Go to **Simulation**, keep the 1C CC Discharge defaults and press "Run simulation"; the summary, charts and safety checks appear within seconds. (6) Walk **Thermal** to see the verdict banner and the 3D heat field, then **Monitor** and press play on the replay dashboard. (7) Finish in **Analysis**, where the new run now sits in the test log beside every earlier experiment, ready to compare and export.

> **Tip**
>
> Loading a library cell resets the pack to 1S1P by design - the library defines cells, not packs. Set your series / parallel targets after loading a cell, and the Connection tab will pre-size the pack from the Design targets automatically.

4. User Guide - The Seven-Step Workflow

This chapter walks through every step of the wizard in the order an engineer would use it, with screenshots captured from a live session that designed a 13S4P pack of Samsung INR18650-25R cells against a 48 V / 10 Ah target. Every control described here exists in the shipped build, and every screenshot is reproducible by following Section 3.3.

4.1 Step 1 · Design - define the cell

The Design step is the single source of truth for the battery. The "Starting point" card offers three ways in: a searchable dropdown of the 24 library cells, a one-tap "Ask copilot to pick a cell" shortcut, or the full Library browser; "Reset" returns to a blank sheet for a fully custom design. Below it, the sheet is organised into labelled cards - Identity and chemistry, Electrical ratings, Dimensions and materials, Temperature limits and durability, Electrolyte, separator and mass, and the 2-RC ECM parameters. Fields that feed the physics are marked as such; documentation-only metadata (cathode chemistry, source reference) is kept separate so it never masquerades as a model input.

Validation is live and explanatory. Missing or inconsistent values are listed as amber chips above the cards ("design name", "capacity (Ah)", "R0 (mOhm)" ...), and the sheet earns a green "Design complete - ready for Connection and testing" badge only when every physics-relevant field is sensible: capacity, nominal voltage, a voltage window whose max exceeds its min, a positive R0, mass, heat capacity, surface area, and both C-rate limits. A live KPI strip computes cell energy, specific energy, 1C current and a pack preview from the current values, and the "Pack targets" fields (for example 48 V / 10 Ah) pre-load the Connection step. Designs can be saved to the database by name and reloaded later from the save slots.

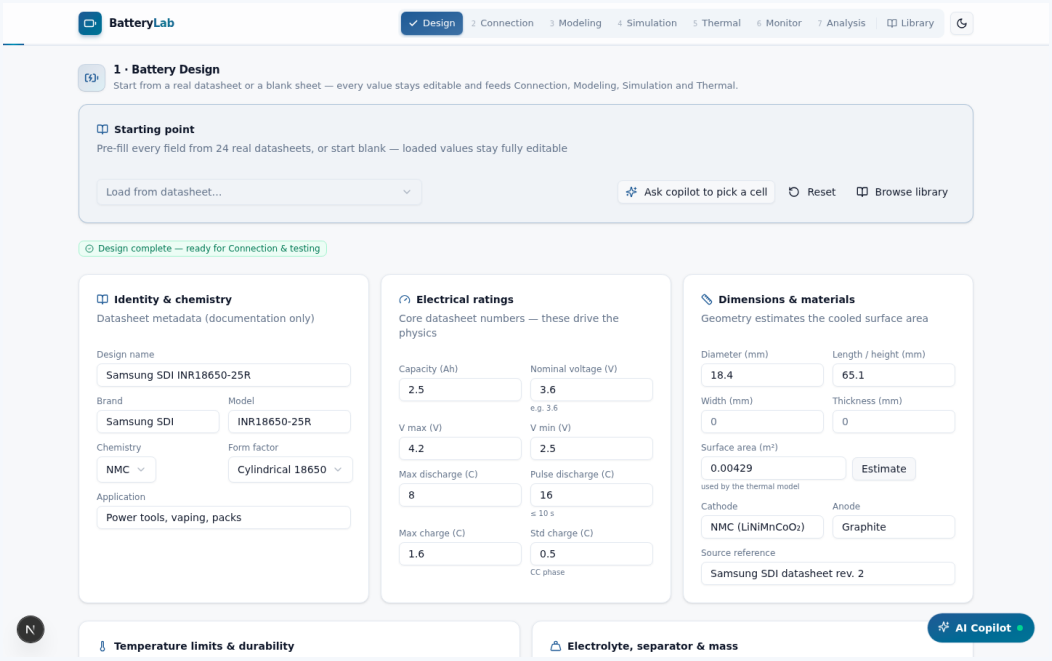


Figure 4-1 · Design step with the Samsung INR18650-25R datasheet loaded; missing-field chips turn into a completion badge

4.2 Step 2 · Connection - size and wire the pack

The Connection step turns one cell into a pack. A banner restates the design intent ("From your design: 48 V and 10 Ah"), and series / parallel steppers with plus-minus buttons, direct value entry and voltage presets (12 / 24 / 48 / 400 V) resize the configuration; the header shows the resulting shorthand such as 13S4P. An eight-card summary recomputes instantly: pack nominal / max / min voltage, capacity, energy, cell count, pack mass and a first-cut gravimetric energy density including a 25 percent structure-and-BMS allowance.

The schematic builder is deliberately manual. Three actions - "Connect parallel" (cells into groups), "Connect series" (groups with nickel strips) and "Connect all" (busbars and the BMS lead) - progressively animate a wiring diagram of the actual S x P grid. The step is gated: "Next: Modeling" stays locked with a hint until the schematic matches the configured S x P exactly, which guarantees that the 3D model, the pack math and the stored configuration can never drift apart. Rebuilding after a size change requires re-wiring, a small price for keeping the visible diagram truthful.

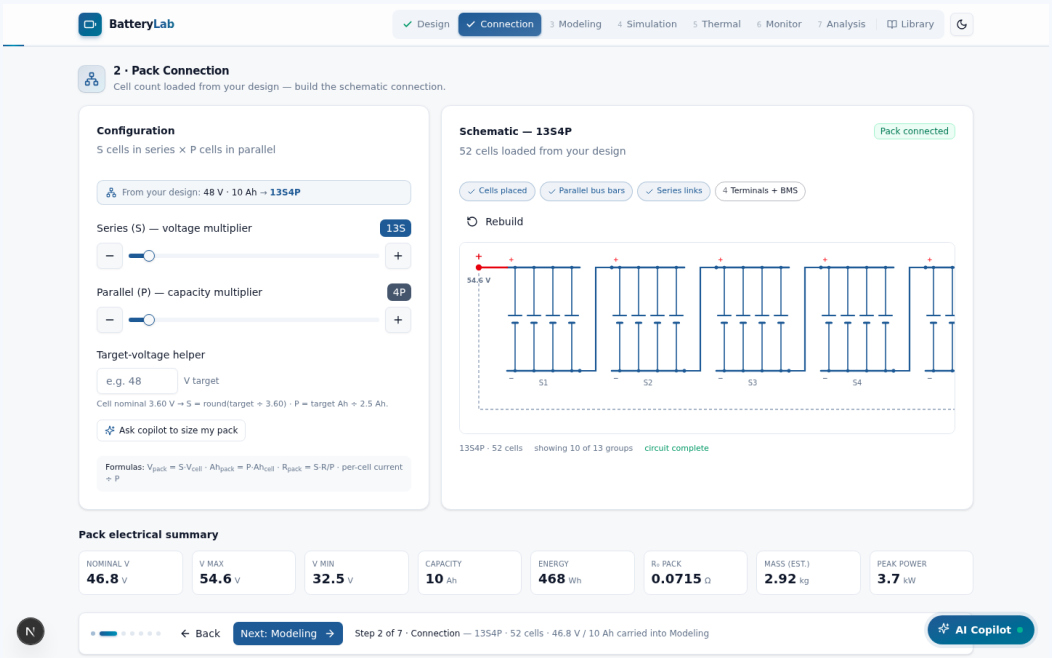


Figure 4-2 · Connection step: 13S4P summary cards and the fully wired schematic (all three build stages complete)

4.3 Step 3 · Modeling - the finished pack in 3D

Modeling shows the mechanical artefact: a fully interactive 3D model of the pack exactly as configured - one cylinder or pouch per cell, parallel groups, nickel strip busbars, serpentine series jumpers, a BMS / CMU board and a translucent enclosure, all rebuilding live when S x P or the cell dimensions change. The view supports orbit, zoom and auto-spin, and two toggles control busbar and enclosure visibility. Polarity is impossible to miss: every cell carries a bright red positive cap on top and a dark negative ring at its base, and the pack-level main output posts are large, colour-matched terminals with washer rings.

A specification strip beneath the canvas reports the mechanical facts - form factor, cell dimensions and mass, total cell count, pack cell mass and energy density - and a legend chip row explains the colour coding (wrapper chemistry colour, nickel busbars, BMS board, red positive, dark negative). All textual overlays are screen-space elements pinned to the viewport corners, so they never intersect the 3D geometry regardless of camera angle; information that changes with the camera is conveyed by colour and geometry instead of floating labels.

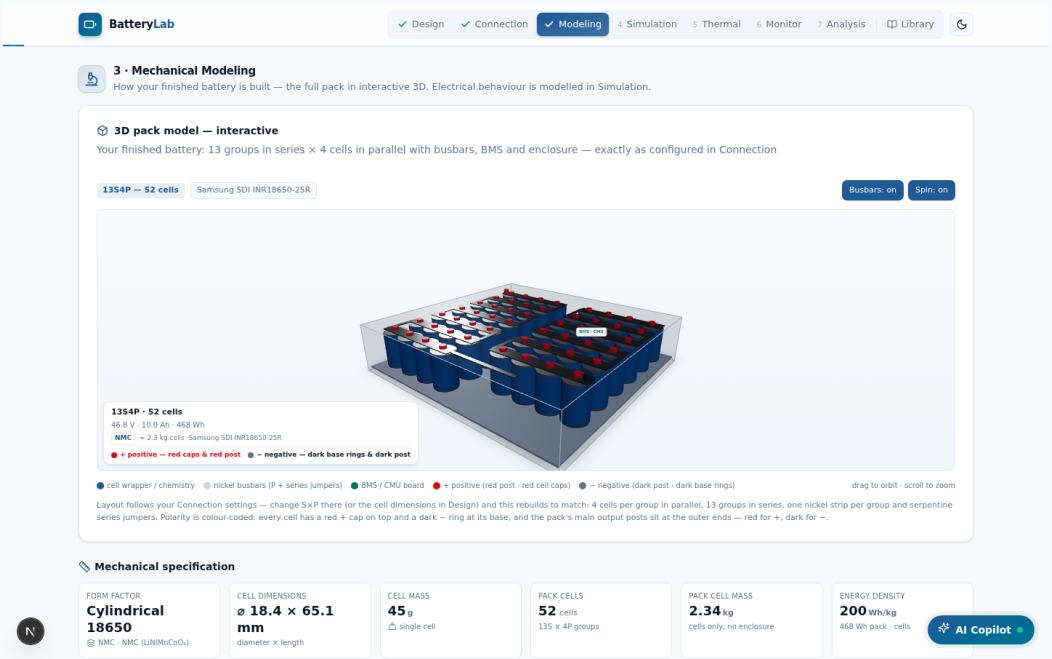


Figure 4-3 · Modeling: interactive 3D pack of the 13S4P design with polarity colour-coding and the bottom-left specification card

4.4 Step 4 · Simulation - run the electrical model

The Simulation step is the lab bench. A profile picker offers five options: CC Discharge (constant current down to V-min), CC-CV Charge (constant current up to V-max, then a voltage taper to C/20), HPPC Pulses (the standard 10 s pulse / 40 s rest / regen impedance sequence with 1C discharge segments stepping 10 percent depth-of-discharge), Drive Cycle (an urban profile of cruise, acceleration, braking regen and stops) and a Custom Sequence builder of arbitrary discharge / charge / rest steps. Test conditions - C-rate, start SOC, ambient temperature, max duration and an optional run name - sit beside the picker with sensible defaults and profile-aware auto-adjustments (HPPC and Drive start below 100 percent SOC so the full sequence fits).

Pressing "Run simulation" posts the design, pack and parameters to the server, which executes the engine (Chapter 7), persists the run and returns series plus summary. The results view opens with a twelve-metric summary grid - throughput, energy, pack energy, average and peak power, SOC window, cell voltage range, temperature rise, heat wasted, cutoff reason - followed by four synchronised charts (cell and pack voltage with V-min / V-max reference lines, SOC, current, temperature and heat generation), a CSV export of the full time series, and a "Safety and limit checks" panel with PASS / WARN / FAIL badges and

human-readable detail lines. Below the charts, the "Live test bench" replays the run as an animated experiment: a cycler console shows voltage and current, electrons flow along the leads, the cell fills and drains with SOC, and gauges track the same numbers, all scrubbable with play / pause and 0.5x to 4x speeds while the charts follow in sync.

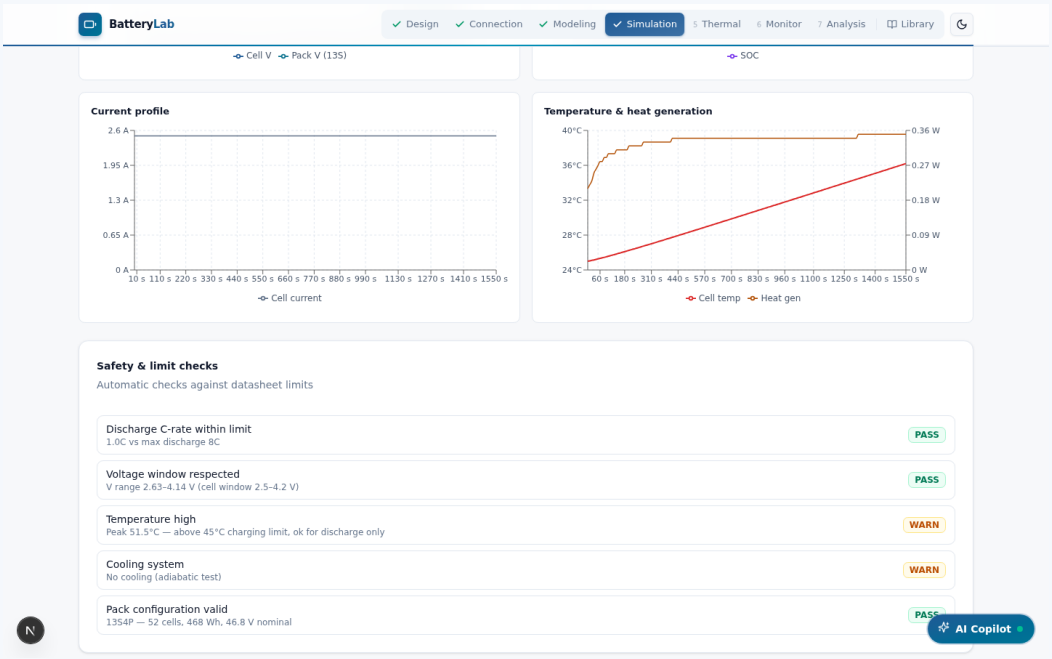


Figure 4-4 · Simulation results: charts for current and temperature plus the automated safety checklist

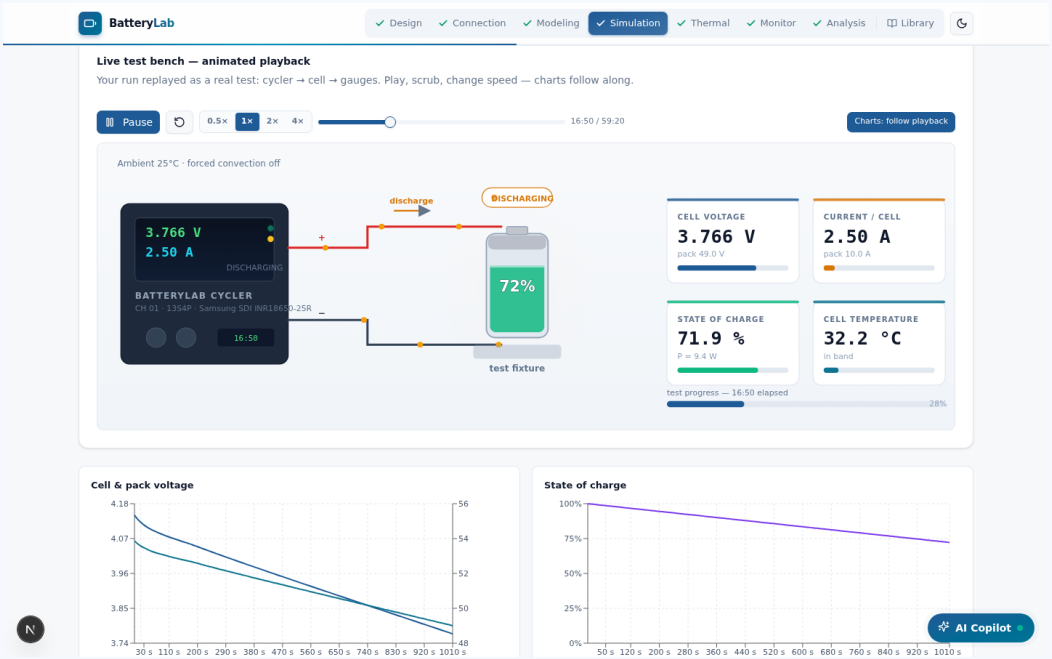


Figure 4-5 · The animated test-bench replay: cycler console, electron flow, SOC fill and live gauges on a shared clock

4.5 Step 5 · Thermal - heat, limits and cooling

Thermal answers the pack-design question that electrical simulation alone cannot: does the heat rejection strategy keep the cell inside its datasheet windows? The left card offers four cooling presets - None (adiabatic), Natural air (h = 8 W/m^{2}K), Forced air fan (h = 35) and Liquid cold plate (h = 150) - plus sliders to fine-tune the convection coefficient and ambient temperature, and a plain-language statement of the thermal model and its limits. Pressing "Run 1C thermal test" executes the coupled electro-thermal model and renders a verdict banner: SAFE, WARN - HOT (fine for discharge, too hot for charging), OVERTEMP or RUNAWAY, accompanied by peak temperature, temperature rise, end temperature and heat-wasted statistics.

Three visualisations follow. A temperature-versus-limits chart overlays the run on the 45 / 60 / 130 degree Celsius reference lines; a Bernardi heat-generation chart breaks down where the watts come from; and the pack-level views extend the single lumped cell into space. The 2D "Live pack thermal map" lays out every cell as a colour-coded tile with an airflow arrow, a scale bar and mean / hottest / coldest readouts. The 3D thermal simulation renders the same field on the actual pack geometry - every cell coloured by modelled temperature, the airflow direction visible, and floating rings marking the hottest (red) and coldest (cyan) cells as the test replays, with temperature values kept in screen-space chips so labels can never overlap.

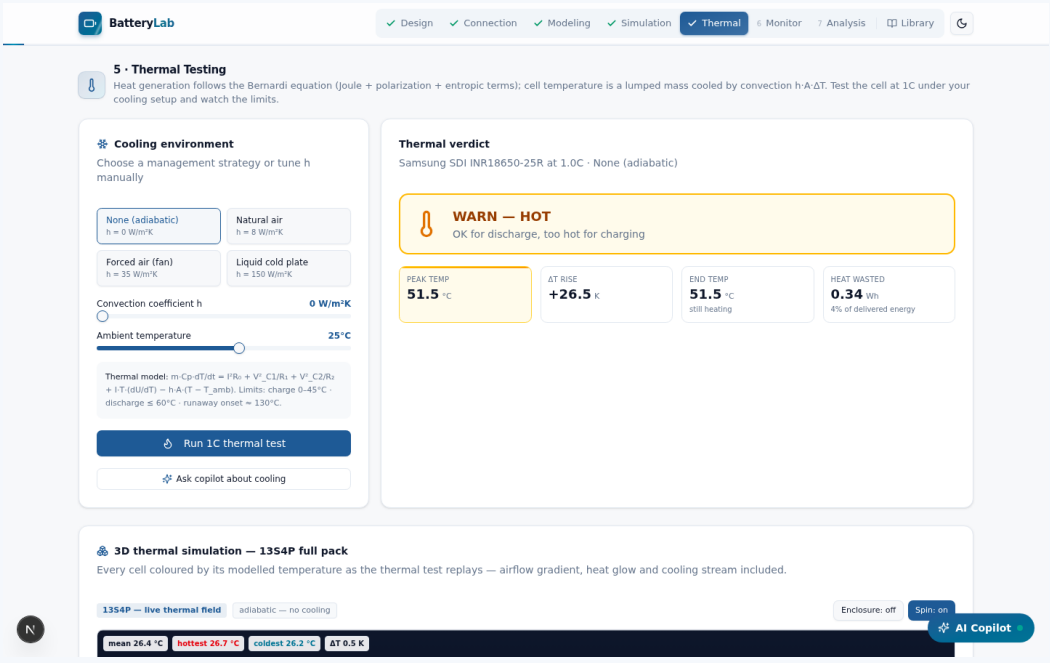


Figure 4-6 · Thermal step: cooling presets, the WARN - HOT verdict for an adiabatic 1C run, and the 3D thermal field below

4.6 Step 6 · Monitor - BMS telemetry, simulated and real

Monitor turns the replayed run into an operations dashboard. Source 1, "Simulation monitoring", replays the latest stored run on the shared playback clock: a status pill (DISCHARGING / CHARGING / IDLE), elapsed test time, eight gauges (pack voltage and current, SOC, pack power, cell temperature with max / min,

temperature spread, cell voltage max / min, remaining energy), a per-cell matrix where each tile shows live voltage and temperature with colour = temperature and a red outline for extremes, contactor states (MAIN+, MAIN- and PRECHARGE with open / closed logic), an alert feed, and streaming strip charts. Playback controls match the Simulation bench - pause, restart, 0.5x to 4x, scrub, and a "Charts: follow playback" toggle.

Source 2, "Live hardware BMS", connects to physical equipment through the Web Serial API - a browser capability available in Chrome and Edge on HTTPS or localhost. Three protocols are supported: CAN bus (J1939-style broadcast frames through a USB-CAN adapter exposing a COM port), RS485 with Modbus RTU register polling, and plain UART ASCII frames via a USB-UART bridge. The panel lists detected ports, opens the stream, and feeds the same gauges, charts and a raw RX frame log from live data; in unsupported browsers the section degrades to a clear notice plus a demo frame stream, so the UI is never dead.

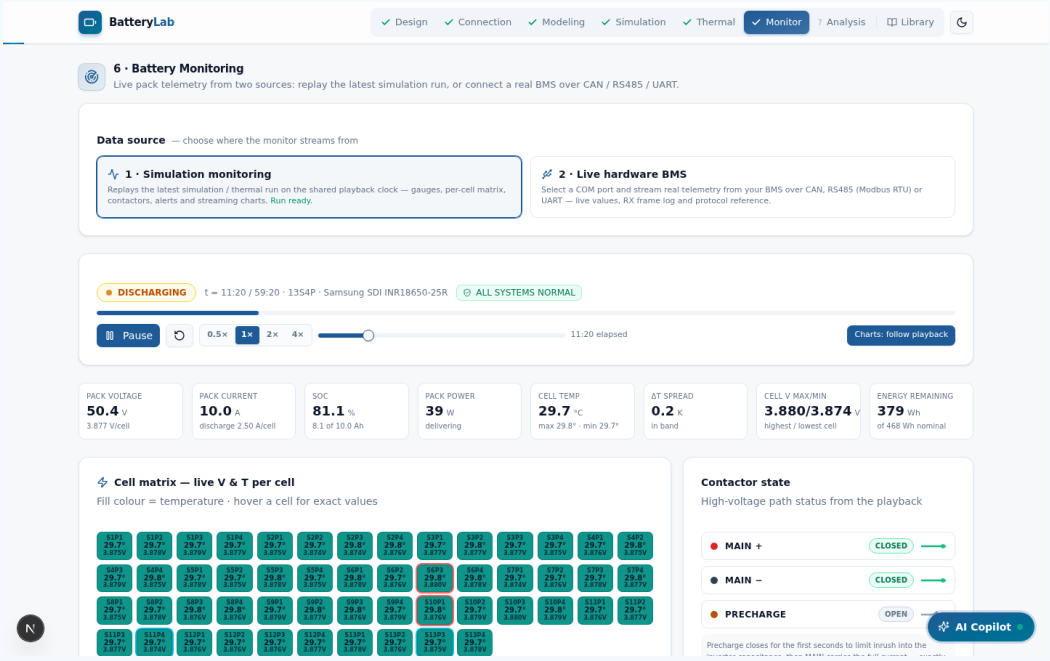


Figure 4-7 · Monitor step replaying the 1C discharge: gauges, per-cell V/T matrix and contactor states

4.7 Step 7 · Analysis - compare, export, conclude

Analysis is the experiment logbook. A program KPI strip aggregates every stored run - total runs, cumulative pack energy, cumulative heat, highest peak temperature and best regen efficiency. The "Final pack - 3D view" card rebuilds the pack from the latest run's stored design and offers two one-click exports: a Spec CSV (all 44 design fields plus pack math) and a Snapshot PNG of the 3D canvas. The test log lists every run with profile, C-rate, ambient temperature, cooling mode and headline results, supporting multi-select of up to four runs for comparison.

Comparisons render as energy and delta-T bar charts and as normalised overlay plots - cell voltage and temperature against a shared 0-100 percent duration axis so runs of different lengths align honestly. Every dataset exports to CSV, runs can be deleted individually or in bulk, and a Refresh action re-syncs with the database. Together with the CSV exports in Simulation, this closes the traceability loop: a reviewer can take any figure in a report back to its raw time-series data.

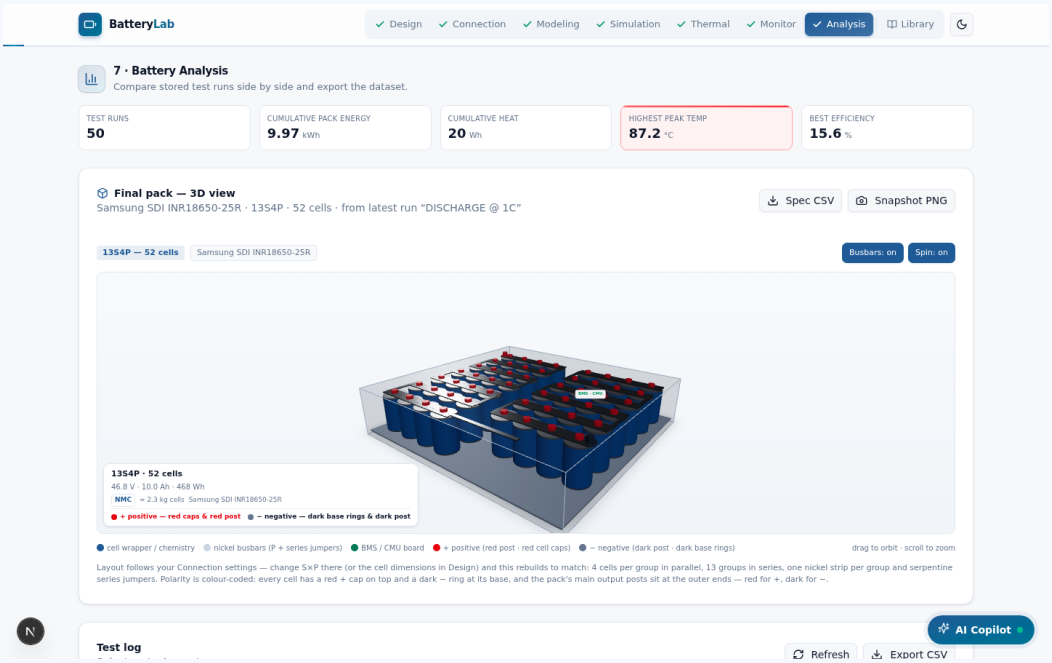


Figure 4-8 · Analysis: program KPIs, the final-pack 3D view with Spec CSV / Snapshot PNG exports, and the test log below

4.8 Library - the datasheet browser

The Library tab is a filterable catalogue of 24 real cells across 17 brands and 8 chemistries, grouped by form factor: cylindrical 18650, 20700 / 21700, 26650 / 32700, prismatic LFP, pouch, coin cells and legacy chemistries. Category chips filter the grid, a search box matches brand and model text, and each card shows the headline specs (chemistry badge, capacity, nominal voltage, max discharge C-rate, mass). Clicking a card opens a detail dialog with the full datasheet sheet and a "Use in Design" action that pre-fills the entire Design sheet - values arrive as an editable starting point, never as a locked preset.

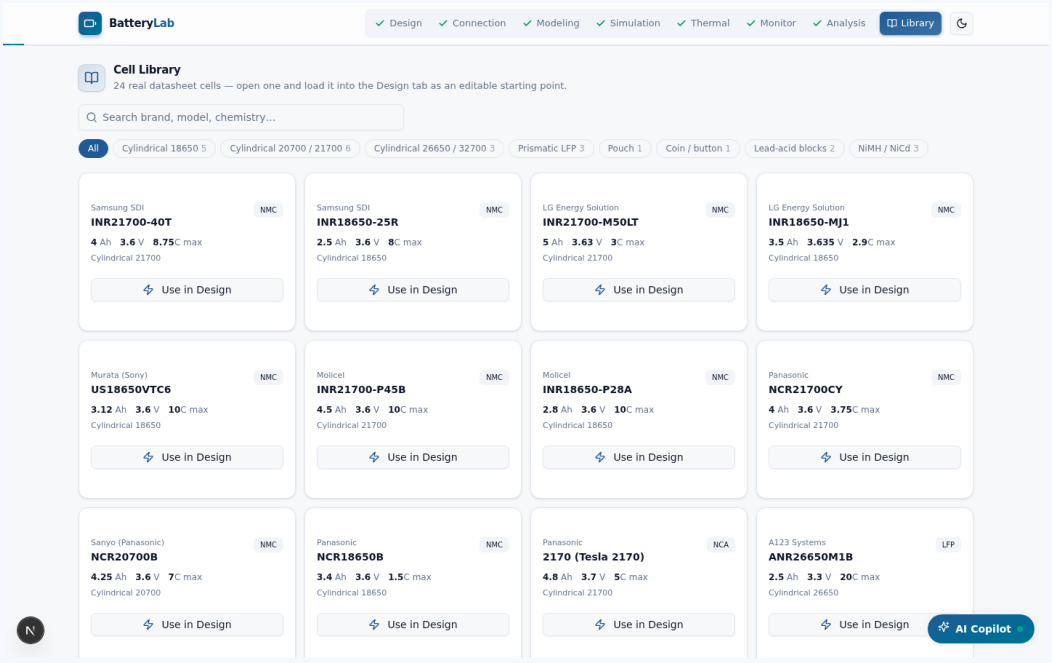


Figure 4-9 · Library: 24 datasheet cells with category filters; the detail dialog pre-fills the Design sheet

5. VoltAI Copilot

VoltAI is a context-aware engineering assistant embedded in every tab, launched from a glowing pill in the bottom-right corner. It is not a generic chatbot bolted onto the page: every message ships a structured snapshot of the live workspace to the server - the active tab, the full cell design with its computed list of validation issues, the pack configuration and targets, the simulation parameters, and the summary plus safety checks of the latest run. The model therefore answers about *this* battery: it can confirm that "heat = 8^2 x 0.011 = 0.704 W" using the user's actual current and resistance, critique a cooling choice against the just-measured temperature rise, or explain why the last HPPC run reported a 4.4 percent regen recovery.

The copilot's signature capability is the **Apply patch**. When the user asks for a concrete change - "recommend a cell for a 48V 10Ah e-bike pack", "make it LFP and double the capacity" - the model ends its reply with a fenced JSON block under the __apply protocol. The client parses the last such block, runs it through a strict sanitiser (Chapter 8.4) that whitelists field names, coerces types, clamps pack sizes to sane ranges and drops anything unrecognised, and then offers an "Apply" button on the chat bubble. One press patches the zustand design and pack state, and the UI - validation chips, KPI cards, 3D model - updates exactly as if the user had typed the values. The conversation itself stays ephemeral in memory; nothing the user says is persisted to the database.

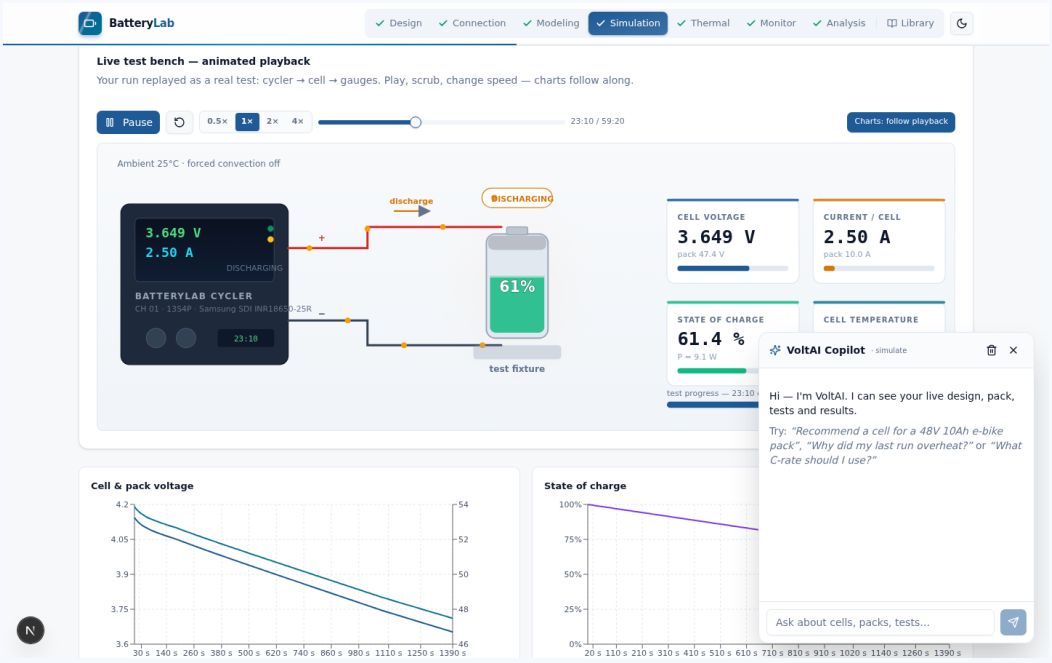


Figure 5-1 · VoltAI opened over the Simulation tab; the panel sees the run in progress and can patch the design on request

> **Safety note**
>
> The copilot is an assistant, not an autopilot: patches only apply after an explicit user press, the sanitiser rejects unknown fields and unsafe values, and every suggestion remains visible in the chat log for audit.

6. System Architecture

BatteryLab follows a deliberately simple three-tier shape: a client-side React application owns all interaction state, a set of thin Next.js API routes validate input and execute the physics engine server-side, and a single SQLite database via Prisma persists designs and runs. Two channels intentionally bypass the server tier: interactive 3D renders entirely on the client GPU through react-three-fiber, and the hardware monitoring mode talks from the browser straight to a physical BMS over the Web Serial API. The figure below shows the tiers and their main modules.

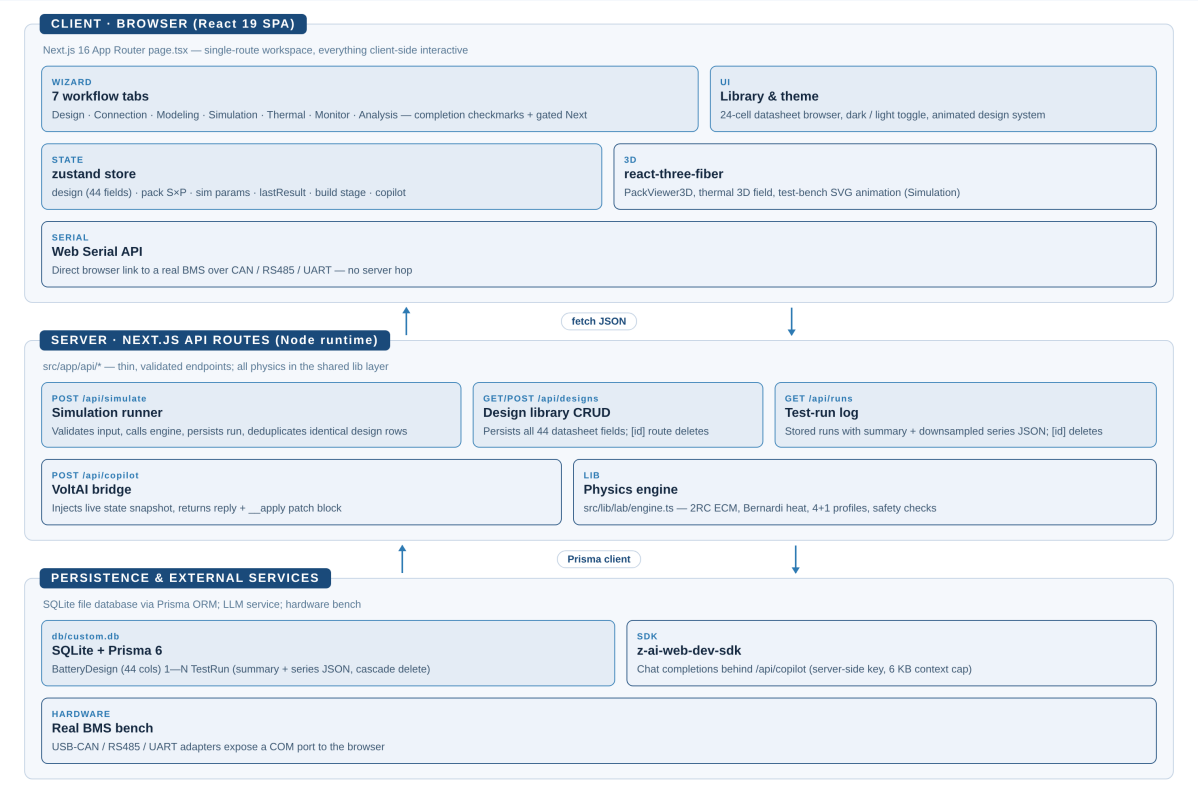


Figure 6-1 · BatteryLab system architecture: browser client, Next.js API tier, and persistence / external services

6.1 Technology stack

Layer	Technology	Version	Role
Framework	Next.js (App Router)	16.1	Single-route workspace, API routes, standalone build
UI runtime	React + TypeScript	19 / 5.x	Client components, strict types across the engine and UI
Styling	Tailwind CSS + shadcn/ui (Radix)	4	Design tokens, dark / light themes, accessible primitives
State	zustand	5	Single lab store: design, pack, sim params, results, copilot
3D	three.js + @react-three/fiber + drei	0.186 / 9.7.0	Pack model, thermal field, canvas snapshot exports
Charts	recharts	2.15	All 2D time-series, overlays and bar comparisons
ORM / DB	Prisma + SQLite	6.11	BatteryDesign and TestRun persistence in db/custom.db
AI	z-ai-web-dev-sdk	0.0.18	Server-side chat completions behind /api/copilot

Layer	Technology	Version	Role
Tooling	bun, ESLint, Prisma CLI	-	Package management, lint, schema push / generate

Table 6-1 · Technology stack (versions as shipped; react-three/fiber pinned to 9.7.0)

6.2 Repository layout

The codebase is compact and flat. The whole user-facing application lives under src/app and src/components/lab, the physics and state under src/lib/lab, and the schema in prisma/. The listing below shows the load-bearing files; generated code (.next, prisma client) is omitted.

Key files

```
src/app/
  page.tsx                 wizard shell: header nav, checkmarks, tab switch, theme, copilot
  layout.tsx               root layout, fonts, theme provider
  globals.css              design tokens, light/dark themes, motion keyframes
  api/simulate/route.ts    POST - run engine, persist run, reuse identical designs
  api/designs/route.ts     GET / POST - design library
  api/designs/[id]/route.ts    DELETE one design (cascades to its runs)
  api/runs/route.ts        GET - stored runs for Analysis
  api/runs/[id]/route.ts   DELETE one run
  api/copilot/route.ts     POST - VoltAI chat with live-context injection
src/lib/lab/
  engine.ts                physics: OCV tables, 2RC + Bernardi sim, safety checks, CSV
  library.ts               24-cell datasheet library and cellToDesign mapping
  store.ts                 zustand store, copilot messaging, AI patch sanitiser
  pack.ts                  deterministic per-cell V/T distribution for pack visuals
  use-playback.ts          shared RAF playback clock for Simulation/Thermal/Monitor
src/components/lab/       18 tab + view components (design, connection, pack-3d, ...)
prisma/schema.prisma      BatteryDesign (44 cols) and TestRun models
db/custom.db              SQLite database file
```

6.3 Client state model

A single zustand store is the client's brain. Its design field is the full 44-field CellDesign; pack carries series / parallel plus the targetV / targetAh set in Design; simParams holds the profile, C-rate, duration, ambient, convection, start SOC, cooling switch and custom segments; lastResult caches the latest SimResponse so Thermal, Monitor and Analysis can replay without refetching. Workflow metadata - build (schematic stage and the S x P it was built for) and visitedSteps - powers the gating and the header checkmarks. The copilot slice keeps the message list and a busy flag, and sendCopilotMessage assembles the context snapshot described in Chapter 5.

Two implementation details are worth recording. First, page.tsx deliberately uses **primitive selectors only**: zustand v5 routes through useSyncExternalStore, and an object-returning selector there caused an infinite-loop regression during development - the fix was to select scalars and derive completion flags per field. Second, loadLibraryCell merges a blank design with the library mapping rather than storing the library object directly, guaranteeing that every loaded design is a plain, fully owned editable sheet even if the library schema evolves.

6.4 3D rendering pipeline

All 3D views share one Canvas configuration: **preserveDrawingBuffer: true** (which is what makes the Analysis "Snapshot PNG" export possible - the buffer is still readable when the download button fires), ContactShadows for grounding, and a capped cell budget (MAX_RENDERED_CELLS) so absurd configurations like 200 x 50 render a representative slice instead of freezing the tab. PackViewer3D derives cell positions from the S x P grid and cell dimensions from the design sheet, so the same component serves Modeling and Analysis with identical geometry.

Materials carry meaning. Each cell's wrapper colour encodes chemistry; the positive terminal is a red cylinder nub on every cell plus a red main post at the pack's negative-of-arc end, while negative is a dark base ring per cell and a dark main post opposite. The 3D thermal view reuses the same pack layout and maps the per-cell temperature distribution from pack.ts onto emissive material colours, with text-free rings marking the hottest and coldest cells - the deliberate architecture of "colour and geometry inside the scene, text in screen-space overlays" eliminates the label-overlap class of bugs entirely.

6.5 Shared playback clock

Simulation's bench, Thermal's scopes and Monitor's dashboard all subscribe to one usePlayback hook. It advances a float position with requestAnimationFrame (a full test replays in about forty seconds at 1x), but re-renders React only when the integer series index changes - roughly ten updates per second regardless of frame rate. Speed multipliers, scrubbing, restart and a "charts follow playback" toggle are part of the hook's API, and the series it exposes is sliced to the playback position so charts genuinely grow as the test proceeds instead of jumping. The result is one coherent notion of "now" across every animated surface in the app.

6.6 Design system and motion

The visual language is a single blue family anchored by the Crystal Blue palette used in this report's own cover: near-black blue surfaces in dark mode, blue-white in light, one gradient accent for brand moments. Motion is implemented exclusively in CSS: keyframes for fade-up, fade-in and scale-in entrances, a .stagger utility that cascades children by 30-80 ms steps, hover-lift shadows on cards, a shine sweep on the logo, and a glow-ping on the copilot launcher. A prefers-reduced-motion media query disables the whole kit. Because nothing depends on a JavaScript animation library, the motion layer adds zero bundle cost and cannot block the render loop that the 3D scenes need.

7. Physics and Simulation Engine

The engine (src/lib/lab/engine.ts, about 670 lines of dependency-free TypeScript) is a coupled electro-thermal model: a 2-RC Thevenin equivalent circuit produces terminal voltage and state of charge, and a Bernardi heat equation with convective cooling produces cell temperature. It runs on the server for every simulation request and is deterministic - the same inputs always yield bit-identical series, which makes replay, comparison and regression testing meaningful.

7.1 Open-circuit voltage model

Each chemistry carries an OCV-SOC anchor table - NMC, NCA, LFP, LTO, LCO, lead-acid, NiMH and NiCd - captured at 7 to 14 SOC points from the flat mid-curve to the knee. ocvSocRaw interpolates linearly between anchors; ocvOfDesign then rescales the curve from the chemistry-native window onto the design's own vmin / vmax so a user who sets a conservative 4.1 V max gets a proportionally rescaled OCV rather than a clipped one. The same tables draw the OCV-SOC chart in the Modeling step, so what the user sees is exactly what the integrator consumes.

7.2 Equivalent circuit and state update

The cell is a 2-RC network: an ohmic resistance R0 in series with two parallel RC branches - R1/C1 for fast charge-transfer polarisation and R2/C2 for slow diffusion. With discharge-positive current i, the state equations integrated by a fixed-step Euler scheme (dt = 1 s) are:

State equations (per cell)

```
dv1/dt = - v1 / (R1*C1) + i / C1        (fast polarization branch)
dv2/dt = - v2 / (R2*C2) + i / C2        (diffusion branch)
dSOC/dt = - i / (3600 * capacityAh)
V_term  = OCV(SOC) - i*R0 - v1 - v2     (discharge-positive convention)
```

Two hard-won details matter. The terminal voltage uses the discharge-positive sign convention V = OCV - iR0 - v1 - v2, which an earlier build had inverted (voltage rose above OCV under load); and the polarisation branches carry no clamping to zero, so negative polarisation during charge emerges naturally - without it, CC-CV taper currents were wrong. In CV mode the engine solves i = (OCV - v1 - v2 - vmax) / R0 each step, forcing the terminal to exactly vmax, and declares the charge complete when the current tapers to C/20 (five percent of capacity).

7.3 Thermal model

Heat generation follows the Bernardi equation split into resistive, polarisation and entropic terms, balanced against convective loss to the ambient:

Lumped thermal node (Bernardi)

```
q_joule = i^2 * R0 + v1^2 / R1 + v2^2 / R2
q_rev   = i * T * (dU/dT)                (entropic / reversible term)
q_loss  = h * A * (T_cell - T_ambient)   (0 when cooling is off)
dT/dt   = (q_joule + q_rev - q_loss) / (mass * Cp)
```

The convective coefficient h comes from the Thermal step's presets (natural 8, forced 35, liquid 150 W/m^2K) or a custom slider; with cooling off the test is adiabatic and the cell integrates pure heat accumulation. Surface area is estimated from the datasheet geometry - cylinder side plus caps for round cells, the full cuboid shell for prismatic and pouch - with a 21700-class fallback when dimensions are absent. The simulation aborts unconditionally at 130 degrees Celsius, the adopted runaway-onset threshold, marking the run "ABORT - thermal runaway onset" so an unsafe configuration is a visible event, not a silent number.

7.4 Test profiles

Profile	Structure	Typical use
CC Discharge	Constant current until V-min or 1 percent SOC	Capacity and runtime characterisation
CC-CV Charge	CC at target C-rate to V-max, then CV hold with taper to C/20	Charger behaviour, charge heat, plating-risk window
HPPC Pulses	10 x (2C pulse 10 s / 40 s rest / 1.5C regen 10 s / 40 s rest / 1C to next 10 percent DOD)	DC impedance R0 / R1C1 / R2C2, regen recovery
Drive Cycle	60 x (cruise 0.35C 60 s / accel 1.3C 20 s / cruise 0.5C 45 s / regen -0.7C 15 s / stop 30 s)	Real-world energy, heat and recovery on an urban loop
Custom	Arbitrary sequence of discharge / charge / rest segments with own C-rates and durations	Reproducing a customer duty cycle or a certification profile

Table 7-1 · Built-in test profiles

7.5 Numerical scheme and outputs

The integrator runs at most 200,000 one-second steps (about 55 hours of simulated time) and downsamples the recorded series by 5, then trims to at most 400 points with the final sample always retained - small enough for JSON storage and recharts, dense enough to keep HPPC pulse edges readable. The summary block aggregates 20 metrics including capacity throughput, net energy, average and peak power, voltage extremes, temperature start / max / end, total heat in watt-hours, the cutoff reason and, for regen-containing profiles, a round-trip recovery percentage (energy returned during regen and charge divided by energy discharged). Pack-level energy and capacity are derived with the S x P factors so cell and pack numbers never blur.

7.6 Safety checks

Check	Thresholds	Verdict logic
C-rate limit	max discharge / max charge from datasheet	PASS within limit; WARN up to 1.5x; FAIL beyond
Voltage window	vmin - 0.02 to vmax + 0.02 V	WARN when the run breaches the cell window
Temperature band	45 C charge limit / 60 C discharge limit / 130 C runaway	WARN above 45; FAIL above 60; FAIL + abort at 130
Charge temperature window	datasheet chargeTempMin/Max	FAIL when charging outside the window (lithium plating risk)
Cooling system	hConv setting	WARN for adiabatic runs with delta-T above 15 K
Pack configuration	S and P at least 1	FAIL otherwise; PASS line restates count, energy and voltage

Table 7-2 · Automated safety checks appended to every run

7.7 Validation performed

The engine was validated against hand calculations during development, and the checks are repeatable. A 1C discharge of a 2.5 Ah cell draws 2.5 A and the recorded throughput matches the integrated SOC; the CC-CV run clamps at exactly 4.20 V on the tested NMC design and completes at the C/20 taper; a 4C adiabatic test aborts at the 130 degree Celsius runaway threshold with the correct event marking; a forced-air run's steady delta-T matches Q/(hA) within rounding; a 13S pack shows the expected 13 x 4.2 = 54.6 V plateau; and the drive cycle's regen recovery lands near 16 percent, consistent with the profile's regen share. During a later session, the 1C rerun produced 3.96 Ah throughput against the same hand-computed expectation, confirming reproducibility after the rebuild.

7.8 Assumptions and limitations

The model is intentionally a design-stage tool, not a laboratory-grade emulator, and its boundaries are worth stating plainly. The OCV-SOC curves are chemistry-generic anchors scaled to the design's window, not per-manufacturer fits; RC parameters are constant with SOC and temperature; the thermal node is lumped (one temperature per cell, with the pack-level spread derived for visualisation rather than solved); ageing, coulombic efficiency below 100 percent and cell-to-cell imbalance inside the electrical integration are out of scope. These choices keep every result explainable in one screen of equations, which is precisely the point of a design-stage platform - and the RC fields, entropic coefficient, heat capacity and area all remain user-editable, so a team with better data can tighten the model without touching code.

8. Data Layer and API Reference

Persistence is intentionally minimal: two Prisma models in one SQLite file. Everything a run needs to be replayed and audited lives in the row itself - the run stores its own summary and downsampled series as JSON strings, and points at the design row it tested through a cascading foreign key.

8.1 Database schema

BatteryDesign carries 44 columns plus identity and timestamps. Its field groups map one-to-one to the Design sheet: identity and datasheet metadata (brand, model, form factor, cathode, anode, separator, electrolyte, application, source reference), electrical ratings (capacity, nominal / max / min voltage), the 2-RC ECM parameters (R0, R1, C1, R2, C2), C-rate limits (continuous, pulse, charge, standard charge), geometry (diameter, length, width, height, thickness), thermal properties (mass, heat capacity, surface area, entropic coefficient), durability (cycle life count and note), temperature windows for charge and discharge, self-discharge, and the stored series / parallel counts. TestRun references its design, records the profile, C-rate, ambient temperature and convection coefficient as columns, and holds the summary and series as JSON strings; deleting a design cascades to its runs.

Model	Key columns	Notes
BatteryDesign	name, chemistry, capacityAh, nominalV, vmax, vmin, r0, r1, c1, r2, c2, max*C, dims, massKg, cpJkgK, areaM2, entropicVperK, temp windows, seriesCount, parallelCount	44 data columns; one row per saved or simulated design; all fields persisted since the Task 14 fix
TestRun	designId (FK, cascade), name, profile, cRate, ambientC, hConv, summary (JSON), series (JSON)	series capped at 400 points; summary holds the 20-metric block incl. cutoff and efficiency

Table 8-1 · Prisma models (db/custom.db)

8.2 API endpoints

Endpoint	Method	Request	Response / behaviour
/api/simulate	POST	design + pack + params inline, or designId to load a saved design	Runs the engine; validates capacity, R0 and S/P; returns series, summary, checks, runId; persists the run best-effort
/api/designs	GET / POST	full design JSON on POST	Lists saved designs / saves one with every field
/api/designs/[id]	DELETE	-	Deletes the design; TestRun rows cascade
/api/runs	GET	-	Stored runs newest-first with summaries for Analysis
/api/runs/[id]	DELETE	-	Deletes one run
/api/copilot	POST	messages (last 14) + context snapshot	VoltAI reply; JSON __apply block embedded when a concrete change is proposed

Table 8-2 · API surface (all JSON; errors return { error } with 400 / 404 / 500)

8.3 The simulate pipeline in detail

POST /api/simulate is the workload hub and repays a close reading. It accepts either an inline design (what the UI sends) or a designId, hydrating the 44 fields from the database with sensible defaults for optional columns. Validation is strict about the fields the integrator cannot survive without: positive capacity, positive R0, and S / P of at least one. After running the engine it tries to persist, and persistence is deliberately clever: if an identical design (same name, chemistry, capacity, voltage window, R0 and S x P) already exists, the run attaches to that row instead of duplicating it - which is why long working sessions produce one design row and many runs. Failures inside persistence are logged and swallowed so a database hiccup can never destroy a simulation result the engineer is looking at.

8.4 Copilot context and patch sanitisation

The copilot route builds a system prompt that fixes VoltAI's persona (concise practical battery engineer, user's own numbers in every calculation, safety flags for overtemperature, C-rate excess, sub-zero charging and the 130 degree Celsius onset), appends the client context snapshot truncated to 6,000 characters, and forwards the last ten conversation turns. The __apply patch protocol is sanitised on the client before anything touches state: AI_DESIGN_KEYS whitelists every settable field, strings are trimmed and capped at 200 characters, numbers must parse finite and non-negative, chemistry must be one of the eight supported values, and pack series / parallel are rounded and clamped to 200 / 50. Anything else is silently dropped - the worst an adversarial or confused model can do is suggest a value the user still has to approve.

8.5 Exports and data escape hatches

Every dataset the platform produces can leave the platform. Simulation exports the full time series (t, cell and pack voltage, SOC, current, temperature, heat generation, OCV, mode) as CSV; Analysis exports the same for any stored run plus the program-level log; Analysis and Modeling export the complete design sheet as a Spec CSV; and the 3D viewport exports a PNG snapshot of the rendered frame thanks to the preserveDrawingBuffer canvas flag. The browser generates all files client-side with Blob downloads, so exports work identically in dev and production without any server round-trip.

9. Frontend Engineering and Quality

9.1 Responsiveness

The layout is mobile-first within a max-width container. The header nav scrolls horizontally on narrow screens instead of wrapping, stat grids collapse from four-across to two-across, the monitor gauges stack, and the 3D viewports resize with their cards. The acceptance bar used throughout development is explicit: at a 390 pixel viewport, document.scrollWidth must equal 390 on every tab - no horizontal scrolling anywhere. That check was exercised repeatedly across the build (and most recently during the design-system pass) on all eight tabs in both themes.

9.2 Accessibility and theming

Semantic structure is respected: nav carries aria-label and aria-current, buttons expose aria-labels where icons stand alone, dialog roles come from the Radix primitives underneath shadcn/ui, and focus states survive the custom styling. The motion kit is fully gated behind prefers-reduced-motion. Colour contrast in both themes was tuned so text passes comfortably against its backgrounds - the body text colour is a deep blue-black rather than pure black, which both softens the page and keeps contrast high. Dark mode is a class-based theme (toggled in-app, remembered across visits) rather than an OS mirror, because battery engineers work in dark control rooms during the day too.

9.3 Verification methodology

Every feature in this report was verified in a real browser before being called done, using a scripted headless-Chrome workflow that drives the app end to end: load a library cell, set targets, wire the schematic, run a simulation, open every tab, toggle themes, resize to 390 pixels and measure scrollWidth, and collect console errors - the bar being zero errors and clean lint and TypeScript output (bunx tsc --noEmit filtered to src/). The screenshots in Chapter 4 come from exactly such a session. Known regressions found this way - unreadable stat-card tints in dark mode, a checkbox double-toggle, flat recharts series from per-Line datasets, an object-selector infinite loop in zustand v5 - were all fixed and re-verified, which is the working

argument for keeping the automated walkthrough as a permanent practice.

9.4 Known constraints

- **Web Serial support** - the live hardware mode needs Chrome / Edge over HTTPS or localhost; other browsers see a graceful notice and a demo stream.
- **Dependency pin** - @react-three/fiber is pinned to 9.7.0; 9.8.x interacts badly with drei on React 19 until upstream fixes land.
- **SQLite scale** - the file database is right for single-user and small-team use; a server deployment with many concurrent writers would graduate to Postgres via a one-line Prisma provider change plus migration.
- **Model scope** - as Chapter 7.8 states, the engine is design-stage physics: chemistry-generic OCV anchors, SOC- and temperature-independent RC, lumped thermal node.

10. Extensibility and Roadmap

10.1 Extension points that exist today

The codebase was shaped for the extensions a battery team is most likely to want. Adding a library cell is a data edit: one entry in library.ts (headline specs plus RC starting values) immediately appears in the Library, the picker and the copilot's knowledge. Adding a chemistry means one OCV anchor table in engine.ts plus one entry in the union type; every downstream component reads the design's window generically. Adding a test profile is a new segment builder in buildSegments - the integrator, summary, safety checks, playback and export chain all consume segments without knowing their origin. The 44-column schema has spare semantic room (sourceRef, cycleLifeNote, application), and the API's design-dedup logic makes bulk imports safe.

10.2 Candidate roadmap

Four features would deepen the platform's engineering value, and each maps cleanly onto the existing architecture. They are listed with the honest scope of work involved, in the order the current user base is most likely to want them.

Candidate	What it adds	Where it plugs in
Cost model	A $/kWh BOM card: cell price benchmarks by chemistry, pack-level cost with structure / BMS overhead, cost per cycle	New packMath-style helper + a Design/Analysis card; library gains a price field
Ageing model	Cycle-life and calendar-ageing curves that degrade capacity and R0 across repeated runs	Engine post-processor keyed off cycleLife fields; Analysis shows capacity fade over program count
Design A/B compare	Score two saved designs against each other on the existing weighted criteria	Reuse the multi-select in Analysis; a scoring table over two design rows
Assumptions panel	A visible assumptions / sources / limitations sheet per run	Static content keyed off engine constants; rendered as a dialog from Simulation and Thermal

Table 10-1 · Candidate features and their integration points

10.3 Deployment notes

The production path is the standard Next.js standalone output: bun run build compiles and copies static assets into .next/standalone, and bun run start boots the server on the host of choice with DATABASE_URL pointing at the SQLite file. Because state is entirely in that file plus the codebase, backup is a file copy and migration between machines is a rsync. For a multi-user deployment the same bundle runs behind a reverse proxy; the two things to revisit are the database (Prisma makes the Postgres move cheap) and the copilot key management (the SDK credential stays server-side today, which is already the correct side of the boundary). The Web Serial mode benefits from HTTPS in production, since browsers reserve the API for secure contexts.

10.4 Closing assessment

BatteryLab demonstrates that a serious engineering workflow - datasheet-driven design, gated assembly, coupled electro-thermal simulation, thermal verdicts, live telemetry and cross-run analysis - can live in a single web page without sacrificing rigour or performance. The physics is documented and editable, the data model is small enough to hold in one's head, and every screen was verified in a real browser against explicit acceptance bars. The platform is ready to support real design conversations today, and the extension points in this chapter are the shortest paths to deepening it tomorrow.